

Universal Log Pre-processing Framework

A team reference brief: what the problem actually asks for, the standards and datasets already out there, the tools to build on, and what would make our solution stand out to evaluators.

ORGANISATION	National Technical Research Organisation (NTRO)
CATEGORY	Software · Miscellaneous
DEADLINE	20 September 2026
CURRENT SCOPE	Perimeter network device logs → standardized, lossless, analytics-ready events

What is actually being asked

Strip the buzzwords and the ask is simple to state: ingest logs in *any* format from *any* vendor, keep the original untouched, convert it into one common structure, and keep a provable link between the two — at very high volume, with no internet access.

THE FIVE THINGS THAT MUST ALL BE TRUE AT ONCE

- **Lossless** — the raw event is preserved exactly, for forensic/compliance use.
- **Normalized** — heterogeneous logs land in one common event taxonomy.
- **Traceable** — every normalized event can be linked back to its raw original.
- **Extensible** — a brand-new log source can be onboarded without writing a new parser from scratch each time.
- **Deployable air-gapped, in a container** — no external calls, no internet dependency, runs anywhere.

Important scoping detail: the background section talks about "modern enterprises" and every log source imaginable (IoT, IAM, cloud, containers). The *Current Scope* section narrows this down hard: build it for **perimeter network device logs** — firewalls, routers, IDS/IPS, proxies. That is our actual MVP target. Everything else in the background is the long-term vision to gesture at in the architecture document, not something we need to build for the demo.

Don't invent a taxonomy — a "universal" one already exists

This is the single most important thing for the team to understand before writing any code: **a standard schema for exactly this problem already exists, twice.** If we invent our own field names from scratch, it will look naive to any evaluator who knows the SIEM space. Our job is to build the ingestion/parsing engine that gets arbitrary vendor logs *into* one of these — that engine is the actual unsolved problem, not the taxonomy.

OCSF — Open Cybersecurity Schema Framework (recommended base)

Backed by AWS, Splunk, and Broadcom/Symantec; now a Linux Foundation project (Nov 2024). It defines 33+ event classes (network activity, authentication, file activity, etc.) organized into six categories, each with required and optional fields, controlled enumerations, and vendor extension points — purpose-built for security telemetry.

Schema browser: schema.ocsf.io · Source: github.com/ocsf

ECS — Elastic Common Schema (secondary reference)

A more general observability/security field schema used across the Elastic Stack. Worth knowing because Elastic and OpenTelemetry announced (April 2023) an intent to converge ECS with OTel semantic conventions into a single standard — relevant context if judges ask about interoperability.

Source: github.com/elastic/ecs

TEAM TAKEAWAY

Pick OCSF as our normalized output schema. Map every parser's output fields onto OCSF classes. Mention ECS/OTel convergence in the architecture doc to show we know the landscape — but don't build two schemas.

Formats and their specifications

FORMAT	WHAT IT IS	KEY DETAIL
Syslog	Transport + framing for almost everything else	RFC 3164 (legacy BSD-style) and RFC 5424 (structured, includes year + timezone). CEF and LEEF both ride on top of syslog.
CEF (ArcSight)	Pipe-delimited header + key=value extension	<code>CEF:Version Vendor Product Version ClassID Name Severity Extension</code> . Pipes and equals signs inside values must be escaped. Full spec: raffy.ch/blog (CEF.pdf) .
LEEF v2 (IBM QRadar)	Syslog header + pipe-delimited LEEF header + tab-separated key=value pairs	Header: <code>LEEF:Version Vendor Product Version EventID Delimiter </code> . Reference: docs.nxlog.co/userguide/integrate/leef.html
JSON / XML / CSV	Structured, self-describing formats	Common for Suricata (<code>eve.json</code>), Zeek/Bro (TSV), and modern cloud/API-driven sources.
Proprietary	Vendor-specific free text	The genuinely hard bucket — this is where template-mining (see Section 6) earns its place.

Where to get real data (this is what most teams skip)

Two very different needs here: (a) diverse *raw log formats* to prove our parser is genuinely universal, and (b) *labelled attack traffic* for the downstream analytics/ML demo. Don't conflate them — most public "intrusion detection" datasets are already flow-extracted CSVs, not raw device log lines.

For raw format diversity — use this first

elastic/beats — filebeat modules

github.com/elastic/beats/tree/main/x-pack/filebeat/module

The single best resource we found. Elastic ships real sample log files *and* real working ingest-pipeline/grok mappings, Apache-licensed, for dozens of perimeter vendors: Cisco ASA/IOS, Palo Alto, Fortinet, Checkpoint, F5, Zscaler, Netscout, and more. Example: `x-pack/filebeat/module/cisco/asa/test/sample.log` has real ASA syslog lines ("Built outbound TCP connection...", "Deny tcp src dmz... by access-group..."). This alone can seed our entire parser test suite and give us working reference mappings to normalize against.

Loghub / Loghub-2.0

github.com/logpai/loghub · github.com/logpai/loghub-2.0

19 real-world system log datasets (distributed systems, OS, servers, standalone software). Loghub-2.0 adds large-scale human-annotated ground-truth templates — the standard benchmark for testing log parsers.

Palo Alto CEF references

github.com/jamesfed/PANOSSyslogCEF · github.com/jcustenborder/palo-alto-syslog-parser

Ready CEF templates and a working syslog parser specifically for PAN-OS firewalls.

Cribl Palo Alto Networks pack

github.com/criblpacks/cribl-palo-alto-networks

A production-grade PAN-OS log reduction/parsing pipeline — useful as a design reference for how a real vendor pipeline is structured, even if we don't use Cribl itself.

Syslog-Generator (Cisco ASA)

github.com/miroslav-siklosi/Syslog-Generator

Synthetic Cisco ASA log generator with labelled/unlabelled and seen/unseen template splits — built for anomaly-detection ML, directly useful for the AI/ML-readiness deliverable.

For labelled attack traffic (analytics/ML demo layer)

CIC-IDS2017

15 labels across 14 attack types plus normal traffic; realistic multi-day simulated enterprise traffic, PCAP + CSV (CICFlowMeter, 80 features).

UNSW-NB15

Nine attack categories, 49 features, generated via IXIA PerfectStorm across a real firewall/router testbed.

Easiest access to both

github.com/rdpahalavan/nids-datasets

Pip-installable, packet- and flow-level, 230M+ packets combined across both datasets.

Note: CIC-IDS2017 and UNSW-NB15 are already flow-extracted (CSV features). They're good for the "threat analytics" demo, not for proving format diversity — use the Loghub / elastic-beats sources for that.

05 — PIPELINE ENGINE

Don't build a parsing framework from zero

Vector (recommended core engine)

Rust-based, MPL-2.0 licensed, genuinely open source — runs fully offline with no vendor account needed, which matters directly for our air-gap requirement. It ingests from dozens of source types and transforms/enriches via VRL (Vector Remap Language), a type-safe language with compile-time error checking, then fans out to multiple destinations at once (SIEM index + data lake, simultaneously).

Logstash grok (secondary / legacy-compatibility path)

The most battle-tested option for irregular vendor formats — grok patterns handle firewall/proxy/custom output without a bespoke parser per format. Heavier (JVM-based) than Vector, so keep it as a fallback path rather than the core engine.

STORAGE LAYER FOR AIR-GAP

- **MinIO** — S3-compatible object storage, runs fully offline, for the raw/lossless store.
- **OpenSearch** — for the SIEM-facing normalized index (avoids Elastic's licensing questions).
- **Parquet on object storage** — for the data-lake side, ML/analytics-ready.

Drain3 — automatic template mining for plug-and-play onboarding

This is our answer to expected-solution bullet (e), "plug-and-play onboarding of new log sources," and it's the part most competing teams will skip.

Drain3 (github.com/logpai/Drain3, IBM Research / logpai) is a streaming online log template miner based on a fixed-depth parse tree. It continuously learns and automatically extracts log templates by masking the variable parts of a message — e.g. it turns "connected to 10.0.0.1" and "connected to 10.0.0.2" into one learned template, "connected to <IP>". It supports IP/number masking, persistence across restarts, and parameter extraction that pulls the variable values back out once a template is known.

Why this matters for the pitch: when a genuinely new, uncoded-for vendor format shows up, Drain3 auto-discovers the template live from a handful of sample lines instead of requiring an engineer to hand-write a grok pattern. A human only needs to map the discovered fields to OCSF once per new template — not write parsing logic per log line, and not per vendor from scratch.

DEMO IDEA

Live demo: feed Drain3 a log format from a vendor we deliberately left out of our hard-coded parser set. Show it auto-clusters a template within a handful of lines, then show a human approving the OCSF field mapping in under a minute. This gives evaluators *evidence* for "reduced parser development effort" instead of a slide claiming it.

How the pieces fit together

```

Raw log/event --> Ingest (Vector agent: syslog / UDP / TCP / file / API listeners)
|
|--> Raw Store (MinIO, S3-compatible, air-gap friendly)
|   - write-once; SHA-256 hash of payload for forensic integrity
|   - tagged with a generated event_id (UUID)
|
'--> Format Detector --> Parser Registry
    | known format? --> existing grok / VRL / regex mapping
    |                               (CEF, LEEF, JSON, Syslog, ...)
    | unknown format? --> Drain3 template miner
    |                               --> proposes new template
    v
    Normalizer --> OCSF-conformant JSON
    - includes event_id (join key back to raw store)
    - includes original_format, source_vendor, ingest_timestamp
    v
    Enrichment (GeoIP, asset inventory, threat-intel
    -- all offline / bundled for air-gap)
    v
    Dual sink: SIEM index (OpenSearch) + Data Lake (Parquet)

```

Each Expected Solution bullet from the PS maps directly onto a piece of this: (a) raw preservation → hash-tagged object store; (b)/(c) source-specific parse + normalize → parser registry + OCSF mapping; (d) traceability → shared `event_id`; (e) plug-and-play → Drain3 + declarative VRL/grok templates; (f)/(g) unified view + SIEM/data-lake integration → OpenSearch + Parquet dual sink; (h) AI/ML-ready → OCSF's typed, enumerated fields; (i) reduced parser effort → Drain3; (j)/(k) air-gap + container → MinIO + OpenSearch + Vector all run fully offline, packaged as a single docker-compose/Helm bundle with no external network calls and pre-bundled GeoIP/threat-intel data.

08 — DELIVERABLES CHECKLIST

What we actually have to hand in

01 Source code link — GitHub/Drive, the working repo.

02 README — setup instructions, must actually run.

03 Architecture document — max 2 pages. Use the flow diagram above as the centerpiece; reference OCSF/ECS explicitly to show awareness of existing standards.

04 Demo video — max 2 minutes. Prioritize showing: lossless raw → normalized traceability, the Drain3 live-onboarding demo, and the air-gap proof (network cut, pipeline still works).

05 Technical presentation — max 5 slides. One slide per: problem framing, schema choice (OCSF), architecture diagram, the Drain3 differentiator, and results/throughput numbers.

Judging checklist

WHAT TO SHOW	WHY IT MATTERS
Raw log → SHA-256 hash → normalized OCSF doc → reconstruct original byte-for-byte	Most student teams normalize and discard the raw, or just <i>claim</i> "lossless." We prove it.
Drain3 live-onboarding a vendor format we never hard-coded a parser for	Direct evidence for "reduced parser development effort" instead of a slide bullet.
An honest throughput number from an actual load test, extrapolated to the "billions/day" claim	Judges will discount unsupported scale claims; a real number with honest extrapolation reads as credible.
Cut network access during the demo video and show the stack keeps working	Directly proves the air-gap requirement rather than asserting it.
Explicit mention of OCSF/ECS in the architecture doc	Signals we know the SIEM landscape rather than reinventing a taxonomy naively.